— Formation Node JS

Planning des journées

- 09h30 : Début
- 10h45 - 11h : Pause
- 12h30 - 14h : Pause déjeuner
- 15h30 - 16h : Pause
- 17h30 : Fin de la journée

Warm Up

- Tour de table
  - Qui suis-je en quelques mots (prénom, années d'XP, appétences, autres …)
- Quelles sont vos attentes pour la formation ?
  - Miro : <Mettre le lien ici>

Sommaire

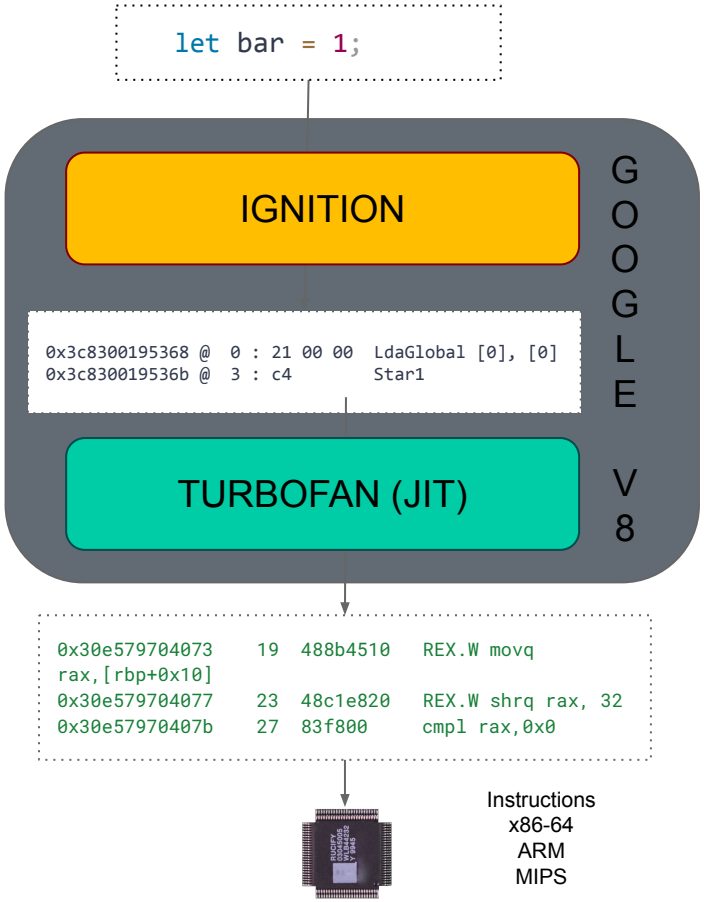
- Le moteur javascript V8
  - EcmaScript
  - Demo : comment Node étends V8
- NodeJS, Javascript côté serveur
  - Github NodeJS
  - Installation
- NodeJS, Les modules
  - CommonJS / ES6
  - Exercice "Utilisons les modules"
- Le gestionnaire de paquets NPM
  - Le client NPM, le registry NPM, le site NPM
  - Exercice "API Meteo"
- Express, un Framework Web pour NodeJS
  - Typescript / Serveur Express / Hot Reload
  - OpenApi, SwaggerUI & SwaggerJSDoc
  - Routes et middlewares
  - Exercice "API Meteo" avec Swagger
  - Test : Tests unitaires / Tests d'intégration
  - Exercice "Backend API Tic Tac Toe"
- Bases de données et ORMs
  - EcmaScript
  - Demo : comment Node étends V8

01 — Le moteur Javascript V8

V8, qu'est ce que c'est ?

V8 est le nom du moteur JavaScript qui équipe Google Chrome. C'est lui qui prend le JavaScript de notre page Web et l'exécute lorsque vous naviguez avec Chrome tandis que Le DOM et les autres API de la plate-forme Web sont fournis par le navigateur

V8 est un moteur JavaScript, c'est-à-dire qu'il analyse le code JavaScript, le transforme en bytecode via son interpréteur nommé IGNITION puis le compile en instructions machines (code machine) via son Compilateur JIT TURBOFAN



La norme ECMASCRIPT

- V8 n'est pas seul, d'autres navigateurs ont leur propre moteur Javascript
    - Firefox possède SpiderMonkey
    - Safari possède JavascriptCore (également appelé Nitro)
    - Edge à l'origine basé sur Chakra mais récemment reconstruit en utilisant chromium et le moteur V8
    - ...
- Tous ces moteurs mettent en œuvre la norme ECMA ES-262, également appelée ECMAScript, la norme utilisée pour standardiser JavaScript

ECMASCRIPT = Standard Javascript

V8, un moteur javascript indépendant du navigateur

- V8 est un projet Open Source de Google
    - https://github.com/v8/v8 https://v8.dev/docs/embed
- Il est écrit en C++, et il est continuellement amélioré
- Il est portable et fonctionne sur Mac, Windows, Linux et plusieurs autres systèmes.
- V8 à été conçu pour pouvoir être intégré (embedded) dans n'importe quelle application en C++ (il est intégré dans chrome par ex.)
- Mais en plus il dispose de hooks C++, il est possible de coder des extensions au standard ECMASCRIPT supporté par V8

DEMO
(Qui a peur du C++ ?)

02 — NodeJS
Javascript côté Serveur

NodeJS

Vous l’aurez compris NodeJS est donc une application C++ qui intègre le moteur V8 de Google
Node est une extension du moteur V8 de chrome auquel Ryan Dahl a ajouté des fonctionnalité destiné à une utilisation côté serveur

- un accès natif aux fonctionnalités du système d'exploitation
  - gestion des fichiers
  - opérations d'E/S
  - sockets réseau
- Mais également d’autres fonctionnalités telles que
  - npm gestionnaire de paquets
  - gestions des modules
  - …

NodeJS est également OpenSource

https://github.com/nodejs/node

outils construits hors de nodeJS
mais faisant partie de nodeJS (v8, npm, uv …)

Javascript
+
(bindings)

C++
(node.h …)

main | 56 branches | 803 tags | Go to file | Add file | Code

.devcontainer	build: add devcontainer configuration	6 months ago
.github	meta: ping TSC for offboarding	last week
android-patches	build: rewritten the Android build system	last year
benchmark	fs: improve error performance of `symlinkSync`	yesterday
deps	deps: patch V8 to 11.8.172.15	2 days ago
doc	doc: update api `stream.compose`	17 hours ago
lib	buffer: remove unnecessary assignment in fromString	7 hours ago
src	fs: improve error performance of `realpathSync`	yesterday
test	test: set `test-structuredclone-*` as flaky	2 hours ago
tools	tools: update comment in `update-uncidi.sh` and `acorn_version.h`	4 days ago
typings	fs: improve error performance of `realpathSync`	yesterday

Installation

https://nodejs.org

node

Apprendre | À propos | Téléchargement | Blog | Docs | Contribuer | Cours

Commencer à écrire... ⌘ K

Exécuter du JavaScript partout

Node.js® est un environnement d'exécution JavaScript gratuit, open-source et multiplateforme qui permet aux développeurs de créer des serveurs, des applications web, des outils en ligne de commande et des scripts.

Obtenir Node.js®

Obtenir une aide à la sécurité
pour les versions Node.js en fin de vie

Node.js est fièrement soutenu par les partenaires ci-dessus et bien d'autres encore.

Create an HTTP Server | Write Tests | Read and Hash a File | Streams Pipeline | Work with Threads

```javascript
// server.mjs
import { createServer } from 'node:http';

const server = createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('Hello World!\n');
});

// starts a simple http server locally on port 3000
server.listen(3000, '127.0.0.1', () => {
  console.log('Listening on 127.0.0.1:3000');
});

// run with `node server.mjs`
```

JavaScript | Copier dans le Presse-papier

Apprenez-en plus sur ce que Node.js est capable d'offrir avec notre Matériel d'apprentissage.

v24.13.0 Dernier LTS | v25.4.0 Dernière Version | Politique de Marque | Politique de confidentialité | Code de conduite | Politique de sécurité

© Fondation OpenJS

03 — NodeJS
Les modules

Modules

- Module : Portion de code réutilisable dont l'existence n'affecte pas accidentellement d'autres portions de code (Modules)
  - Lorsque node a été créé (2009) les modules n’existaient pas dans la standard ECMAScript (ES5), la notion de module à été introduite dans ES6 (ECMAScript 6 en 2015)
- NodeJS implémente les modules selon le standard COMMONJS Modules
- Un module est simplement un fichier javascript (.js) et il est possible d’appeler un module à partir de l’instruction require("./monFichier.js") ou simplement require("./monFichier")

Exercice : "Découvrons les modules"

15 minutes

- Dans un dossier créez le fichier monModule.js

```
var iss = 'Hello petit module !';

console.log('Je suis dans le module, T.Pesquet');
```

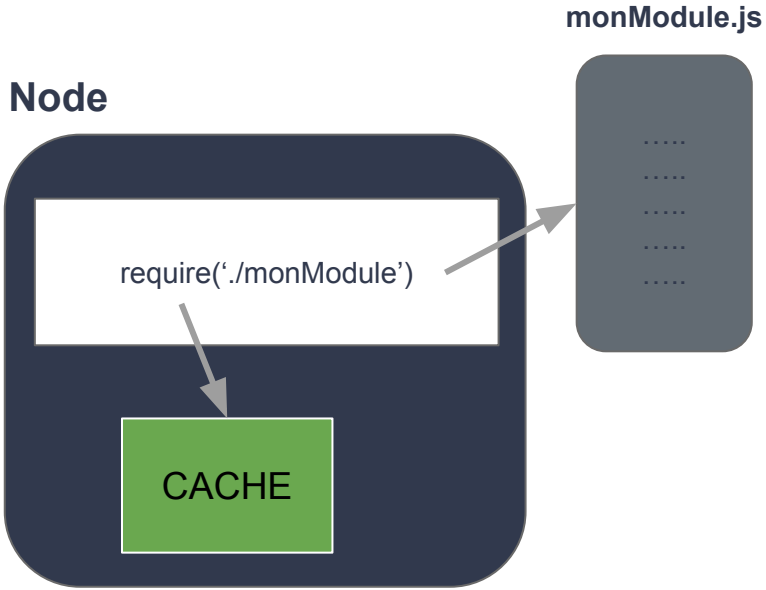
- Ouvrez node et importez le module à l'aide de l'instruction require
- Essayer d'afficher la variable iss

```
console.log(iss);
```

- Que se passe t'il ? (**4 choses normalement**)
- Parlons en !

Chargement des modules

- Node comporte un cache, les modules ne sont chargés qu'une seule fois (exécutés séquentiellement qu'une seule fois)
- Les fois suivantes ils sont récupérés depuis le cache de node
- Si rien n'est prévu au niveau du module pour décrire ce qu'il retourne, le module retourne un objet vide
- Afin de permettre d'organiser son code en dossiers, on peut créer un fichier index.js dans un dossier (Ex : monModule/index.js)



Objets retournés par un module (Exports)

- L'instruction module.exports permet de retourner un objet ou une primitive

```
function capitales(mot) { return mot.toUpperCase()}

module.exports = capitales
```

- Il est ensuite possible de charger ce module dans une variable depuis node ou un autre module

```
var metEnMajuscules = require('./monModule')

metEnMajuscules("Javascript")
```

Exercice : “Utilisons les modules”

15 minutes

- Créer deux modules formatAmericain et formatEuropeen sous forme de dossiers (./formatAmericain/index.js et ./formatEuropeen/index.js)
- Dans chacun de ces modules créez une fonction afficher(nombre) qui reçoit un nombre et qui renvoie respectivement :
    - `new Intl.NumberFormat('en-US').format(nombre)`
    - `new Intl.NumberFormat('fr-FR').format(nombre)`
- Créer un module principal app à la racine sous la forme d’un fichier app.js suivant :

```
/* Mettez vos requires ici */

const nombre = parseFloat(process.argv[2]);

if (isNaN(nombre)) {
console.error("Veuillez fournir un nombre valide en tant qu'argument.");
process.exit(1);
}

console.log('Format Européen:', FormatEuropeen.afficher(nombre));
console.log('Format Américain:', FormatAmericain.afficher(nombre));
```

Modules natifs (Core Modules)

https://nodejs.org/api/

Node fournit beaucoup de modules natifs permettant d'effectuer un très grand nombre de fonctionnalités (utils, path, http, zlib ...)

On peut les utiliser de la même manière que les modules présentés précédemment (require) , sauf qu'il ne faut pas indiquer ni de chemin ni d'extension

```
const https = require('https');

const PARIS_WEATHER_URL = 'https://wttr.in/Paris?format=3';

function afficherMeteo() {
  https.get(PARIS_WEATHER_URL, (res) => {
    // l'api https native est complexe à lire ...
  });
}

module.exports = afficherMeteo;
```

ES6 (2015) et les modules

- Les choix d'implémentation des modules selon les standard CommonJS au sein de node à posé des problème à l'adoption du standard ES6 (qui intègre le standard ES6 Modules), c'est d'ailleurs un point de dette technique auquel Deno réponds (parmi d'autres)
- Pour utiliser le standard ES6 Modules voici les solutions
  - Mettre "type"="module" dans package.json (on en parle plus tard)
  - Renommer les fichiers .js en .mjs
  - Si on utilise "type"="module" on peut quand même utiliser les .js en les renommant .cjs

Modules ES6

Les modules ES6 sont bien plus cool à utiliser, voici les éléments à connaître

- export pour exporter au lieu de modules.exports =

```
export const maConstante = "Hello"

export const maFonctionUtilitaire = () => { console.log("Hello!") }

export default maLibrairie = { ajouter: (a, b) => a + b, multiplier: (a, b) => a * b }
```

- import pour importer des exports depuis un autre module
- import * existe aussi comme syntaxe

```
import librairie, { maConstante, maFonctionUtilitaire } from './monModule'

librairie.ajouter(2, 3)
librairie.multiplier(2, 3)
```

04 — Le gestionnaire de paquets NPM

NPM, L’accès au plus grand écosystème de logiciels au monde

- NPM : Node Package Manager
  - Package : Du code délivrant une ou plusieurs fonctionnalités utiles, pratiques, …
  - Gestionnaire de Paquets : Logiciel qui automatise l’installation et la mise à jour de paquets (Packages)
    - Gestion des versions, Gestion des dépendances, …
  - Bien que le "registry" npm contient une grande partie de packages Open Source tous ne le sont pas forcément
  - La publication de paquets open source sur npm est gratuite, la publication et l'hébergement de paquets privés nécessitent un abonnement payant

NPM, Utilisation

- Pour utiliser NPM, il faut avant tout créer un projet, ce projet peut être un package, un projet node, react… Il est créé à l'aide de la commande `npm init` .
- Le projet (package) npm est alors représenté par le fichier package.json
- Il est alors possible d'installer des packages du registry npm dans votre projet avec `npm install <package>` .

```
{
  "name": "meteo",
  "version": "1.0.0",
  "description": "Meteo à Paris",
  "main": "app.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "ymedaghri@gmail.com",
  "license": "ISC"
}
```

Version du package (projet)

Point d'entrée qui s'exécute si on fait `node .`

Section scripts de notre projet > **npm run test**

MIT en moins verbeux

NPM, le client et le "registry"

- Le terme NPM peut signifier deux choses
  - Le gestionnaire de paquet incorporé dans Node
    - Il permet d'installer les paquets d'autres développeurs
  - Le registre (repository global) contenant les paquets produits par les développeurs du monde entier https://www.npmjs.com/
    - Ouvert à tout le monde
    - Contient les paquets que le client npm installera sur votre projet
    - Contient tout un ensemble d'information sur les paquets (auteur, github, documentation, utilisation, licence, stats ...)

Exercice : "Api Meteo"

15 minutes

- Créez un dossier meteo et lancez un npm init, donnez comme point d'entrée le fichier `app.js` par exemple
- Ajoutez cette ligne au fichier package.json afin de pouvoir utiliser les modules ES6 : `"type": "module"`
- Allez voir sur npm (https://www.npmjs.com/) pour savoir comment marche le paquet `node-fetch` et installez le dans votre projet meteo
- Dans le fichier `app.js` appelez l'url suivante puis affichez là dans la console
  `const WEATHER_URL = 'https://wttr.in/Paris?format=3'`
- Vous êtes joueur ? Modifier le code pour pouvoir passer la ville en paramètre lors de l'exécution `node . Hawaï` , `node . Londres` , `node . Hossegor` ...

SEMVER, Semantic Versionning

- Versionning
  - Les paquets installés via NPM ont des numéros de versions qui permettent de les identifier
  - Ces versions doivent également permettre de savoir si une version peut contenir des changements cassants par exemple (breaking changes)
- L'idée du Semantic Versionning est que l'observation de la manière dont est écrit la version transmets certaines informations comme le fait qu'une version soit cassante par exemple

MAJOR.MINOR.PATCH

2.4.5

Des bugs ont été fixés, ton code continuera de marcher sans problèmes

Des nouvelles fonctionnalité ont également été ajoutées, ton code continuera de marcher sans problèmes

Des changements majeurs ont été introduits, ton code pourrait être impacté

node_modules, package.json, package-lock.json

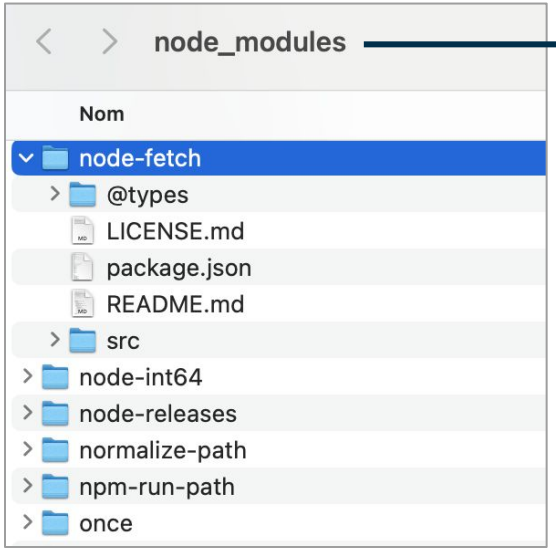
```
"dependencies": {
  "node-fetch": "^3.0.0"
},
"devDependencies": {
  "jest": "^29.7.0"
}
```

npm i –save node-fetch

npm i –save-dev jest

Tout se retrouve dans node_modules y compris les dépendances transitives

^29.7.0 : Met à jour le minor et le patch en cas de npm update
~29.7.0 : Met à jour le patch uniquement en cas de npm update
29.7.0 : Ne met rien à jour en cas de npm update



Le dossier node_modules n'est jamais pushé sur le repo, on peut le supprimer à volonté.
Il est recréé automatiquement via npm i (i ou install)

npm install --only=prod : les packages devDependencies ne seront pas installées dans node_modules

- Réduction significative de la taille de déploiement en prod
- Sécurité : Surface d'attaque réduite

On commit package-lock.json afin d'être iso prod lors d'un npm i , lorsque l'on souhaite monter les versions on fait un npm up en local, on test puis on push le nouveau package-lock.json

05 — Express, un framework Web pour NodeJS

Express JS

Il est parmi les framework NodeJS les plus populaires avec 64,000 étoiles sur GitHub et ses **29,6 Millions** de téléchargements **par semaine** sur Npm, et est largement adopté dans l'industrie

Néanmoins, il en existe beaucoup d'autres :

- NestJS : 64,400 npm 6800
  - Son analogie avec Spring Java le rend attrayant pour de nombreux développeurs
- Koa : 34,400 npm 1,582,172
  - Apprécié pour sa modernité et sa simplicité
- HapiJS : 14,400 npm 829,240
  - Framework solide et bien considéré
- ...

Serveur Express en Typescript

- TypeScript présente des avantages indéniables en matière de qualité du code, de prévention des erreurs et d'efficacité du développement
  - Typage
  - Autocomplétion et Refactorisation
  - Interfaces et Types Personnalisés
  - Intégration des IDE
  - ...
- Nous allons voir comment configurer express avec Typescript et mettre en place le hot reload lors des modifications de code pour améliorer la DX (developer experience)

Typescript & Nodemon

```
"main": "dist/index.js",
 "scripts": {
   "build": "tsc",
   "start": "node dist/index.js",
   "dev": "nodemon",
   "test": "echo \"Error: no test specified\" && exit 1"
 },
 "nodemonConfig": {
   "watch": [
     "."
   ],
   "ext": "ts",
   "ignore": [
     "**/*.spec.ts"
   ],
   "exec": "ts-node index.ts"
 },
 "dependencies": {
   "express": "^4.18.2"
 },
 "devDependencies": {
   "@types/express": "^4.17.20",
   "@types/node": "^20.8.7",
   "nodemon": "^3.0.1",
   "ts-node": "^10.9.1",
   "typescript": "^5.2.2"
 }
```

package.json

- npm init pour créer le fichier package.json
  On ne mets pas "type":"module", car on configurera ES6 Modules dans typescript
  npm i -D typescript @types/express @types/node ts-node nodemon
- Le -D est l'équivalent de --save-dev

Configuration nodemon (hot reload)

C'est la seule dépendance de production npm i express

nodemon c'est pour le hot reload
Le reste pour typescript, la compilation et le typage

- npx tsc --init pour créer le fichier tsconfig.json

```
{
 "compilerOptions": {
   "outDir": "./dist",
   "target": "es2016",
   "module": "commonjs",
   "esModuleInterop": true,
   "strict": true,
   "skipLibCheck": true
 },
 "include": [
   "**/*.ts"
 ],
 "exclude": [
   "node_modules"
 ],
}
```

tsconfig.json

Dossier de destination des js transpilés

Modules des js transpilés CommonJS (node)

Interopérabilité entre CommonJS et ES6 Modules lorsque on travaille avec des bibliothèques qui ne sont pas des modules TypeScript ou ES6 natifs

Fichiers à inclure à la compilation

Fichiers à exclure de la compilation

Code du serveur Express

```
import express, { Express, Request, Response } from 'express'

const app: Express = express()

app.get('/', function (req: Request, res: Response) {
  res.send('Hello Express !')
})

app.get('/api', (req: Request, res: Response) => {
  res.json({ message: "Hello Express !" })
})

const port = 3000
const server = app.listen(port, () => {
  console.log(`⚡[server]: Server runs at http://localhost:${port}`);
});
```

Mise en place de Swagger et SwaggerJSDOC

- `npm i swagger-ui-express swagger-jsdoc` : client swagger et jsDoc en dépendances **prod**
- `npm i @types/swagger-ui-express @types/swagger-jsdoc -D` : types en dépendances **dev**

```
import express, { Express } from 'express'
import swaggerUi from 'swagger-ui-express'
import swaggerJsdoc from 'swagger-jsdoc'
import registerPersonRoute from './routes/person'

const app: Express = express(), port = 3000
const options = {
  definition: {
    openapi: '3.0.0', info: {title: 'Mon Api', version: '1.0.0', description: 'Une API'},
    servers: [{url: `http://localhost:${port}`}]
  },
  apis: ['./routes/*.ts']
}
const specs = swaggerJsdoc(options);

registerPersonRoute(app)
app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(specs));
const server = app.listen(port, () => {console.log(`⚡Serveur http://localhost:${port}`);});
```

Swagger JSDOC : https://swagger.io/docs/specification/about/

```
import { Application, Request, Response } from 'express';
export default (app: Application) => {
/**
* @swagger
* /person/{nom}:
* get:
* summary: Recevoir un Hello World <Nom>.
* description: Toujours le fameux Hello world.
* parameters:
* - in: path
* name: nom
* schema:
* type: string
* required: true
* responses:
* 200:
* description: un message simple.
* content:
* application/json:
* schema:
* type: object
* properties:
* message:
* type: string
*/
app.get('/person/:nom', (req: Request, res: Response) => { res.json({ message: `Hello World ${req.params.nom} !` })})};
```

Les Routes dans Express https://expressjs.com/fr/guide/routing.html

- Le Routage fait référence à la définition de points finaux d'application/endpoints (URI) et à la façon dont ils répondent aux demandes client
- Voilà ce que dit la documentation
  - Express prend en charge les méthodes de routage suivantes qui correspondent aux méthodes HTTP : get, post, put, head, delete, options, trace, copy, lock, mkcol, move, purge, propfind, proppatch, unlock, report, mkactivity, checkout, merge, m-search, notify, subscribe, unsubscribe, patch, search, and connect
- Bon on ne va pas se mentir, vous ne croiserez que POST, PUT, GET, DELETE, parfois PATCH et beaucoup plus rarement OPTIONS et HEAD, quand aux autres probablement jamais

```
app.get('/', function (req, res) { res.send('Hello'); })
app.get('/ab+cd', ...
app.get(/.*fly$/, ...
app.get('/person/:nom', (req: Request, res: Response) => {
    res.json({ message: `Hello World ${req.params.nom} !` });
});

res.send ...
res.json ...
res.sendFile ...
```

Attention ce n'est pas tout à fait une regex (lire la doc)

Pas de surprises, c'est une vraie regex

Paramètres de l'url **:nom** → req.params (différent de la queryString **? et &** où on utilisera alors req.query

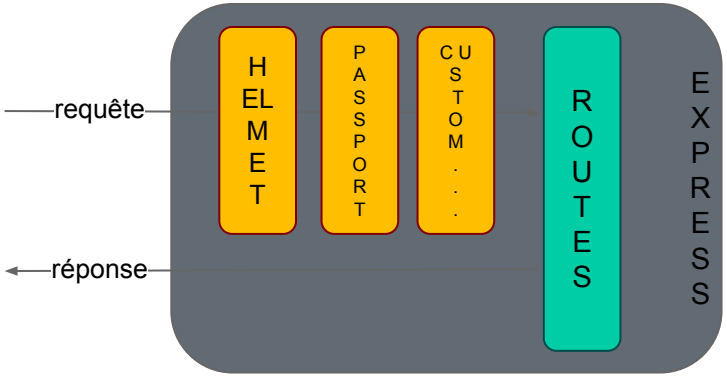
Envoie une réponse de type text/html

Envoie une réponse de type json

Envoie une réponse sous forme de flux d'octets

Les Middleware : Un système de plugin entre la requête et la réponse

- Lorsque le serveur Express reçoit des requêtes HTTP qu'elle soient faites avec différents verbes (POST, GET, PUT, DELETE…) il existe souvent des mécanismes transverses que l'on souhaite appliquer à ces requêtes, les middlewares sont faits pour cela
  - Exemple : Mes requêtes envoient un **Header Authorization** avec un prefix **Bearer** suivi de mon token JWT afin que côté serveur mes permissions et la validité de mon token soient validées
    Clairement on ne va pas faire du code copier collé dans chacune de mes routes, un middleware sera choisi pour cela
  - Middlewares courants : **SwaggerUI**, Helmet(entêtes http contre les vulnérabilités courantes), rate-limit, passeport(JWT) …
- Le site de Express contient plusieurs exemples d'utilisation de middlewares dans sa section ressources
  https://expressjs.com/fr/resources/middleware.html




```
function monMiddlewareJWT(req: Request, res: Response, next:
NextFunction) {
  const bearerHeader = req.headers['authorization'] as string
  . . . jwt.verify . . .
  if (err) { res.sendStatus(403); } else { next(); }
}
app.use('/protected', monMiddlewareJWT);
```

Paramètres, QueryString et POST Body

- Il existe plusieurs manière de transmettre de l'information via des requêtes HTTP
  - les paramètres d'url : **/users/123** → req.params
  - la query string : **/search?q=john** → req.query
  - Request Body (Principalement utilisé avec les méthodes POST, PUT, PATCH)
    - Types de contenu courants: (application/json, application/x-www-form-urlencoded, multipart/form-data (souvent pour le téléchargement de fichiers), text/plain
  - req.body + généralement middleware (body-parser, express.json(), express.urlencoded())
  - les headers : Authorization: Bearer YOUR_TOKEN → req.headers
  - Cookies → req.cookies + middleware cookie-parser
  - Form Data : dans le body comme une chaîne de requête (key=value&key2=value2) → req.body
  - HTTP/2 Server Push et WebSockets : Hors Scope (cf Node concepts avancés)

Exercice : "Meteo avec Axios et SwaggerUI"

30 minutes

- Créer un projet Express et mettre en place SwaggerUI sur un projet en Typescript
- Créer une route GET `/meteo/:ville` et appeler l'url suivante en utilisant la librairie axios (vous pouvez vous aider de npm pour voir comment axios fonctionne)
- L'url du service à appeler est la suivante : `https://wttr.in/Paris?format=j1`
- Passez le paramètre ville à l'appel de l'api
- N'hésitez pas à demander de l'aide dès que vous le souhaitez

- Tester avec Swagger

Les Tests

Lorsque l'on souhaite écrite des tests on peut utiliser plusieurs librairies disponibles sur NPM pour cela : Mocha, Jest, Supertest, MockServiceWorker (MSW), Vitest ainsi que plusieurs librairies d'assertions telles que chai, sinon …

- Nous utiliserons **Vitest** qui est une excellente librairie qui encapsule les fonctionnalités de Jest mais également les assertions Chai, …
- Installation
  - `npm i -D vitest`
  - Puis dans la partie scripts de package.json

    ```
    "test": "vitest"
    ```
- Nommez votre fichier monModule.test.ts
- Vous pouvez alors lancer les tests via `npm run test`
- Si toutefois vous préférez Jest
  - `npm i -D jest ts-jest @types/jest`
  - `npx ts-jest config:init`
  - Package.json `"test": "jest"`

Ecrire un test, un premier pas vers le TDD

Pour écrire une suite de test, on utilise `describe` et on ajoute les tests avec `it` ou `test`

Si vous utilisez vitest ajoutez `import { expect, describe, test } from 'vitest'`

```
describe('Tic Tac Toe', () => {
   it("Doit prendre le centre si il est vide", () => {
      // Given
      const board = [0, 0, 0, 0, 0, 0, 0, 0, 0]
      // When
      const updatedBoard = playComputer(board)
      // Then
      expect(board).toEqual([0, 0, 0, 0, 2, 0, 0, 0, 0])
   })
})
```

Il est aussi possible de créer des tests paramétriques

```
test.each([
["première", [2, 2, 0, 0, 0, 0, 0, 0, 0], [2, 2, 2, 0, 0, 0, 0, 0, 0]],
["deuxième", [0, 0, 0, 0, 2, 2, 0, 0, 0], [0, 0, 0, 2, 2, 2, 0, 0, 0]],
["troisième", [0, 0, 0, 0, 0, 0, 2, 0, 2], [0, 0, 0, 0, 0, 0, 2, 2, 2]],
])('Doit effectuer un mouvement gagnant sur la %s ligne si possible',(_, board, result) => {
// When
const updatedBoard = playComputer(board)
// Then
expect(board).toEqual(expectation)    });
```

Et les tests d'intégration ?

Lorsqu'on veut tester son api de manière automatisée en appelant les endpoints sur le serveur express, il est possible d'écrire des tests d'intégrations utilisant par exemple le framework **supertest**

- Installation
  - **npm install supertest**
  - **npm install -D @types/supertest**

```
import { expect, describe, test } from 'vitest';
import request from 'supertest';
import app from '../../app'

describe(`Tests d'API`, () => {
  test('should return 201 OK for /new-game', async () => {
  // When
  const response = await request(app).post('/new-game');
  // Then
  expect(response.status).toBe(201);
  expect(response.type).toBe('application/json');
  expect(response.body).toHaveProperty('board', [0, 0, 0, 0, 0, 0, 0, 0, 0]);
}); });
```

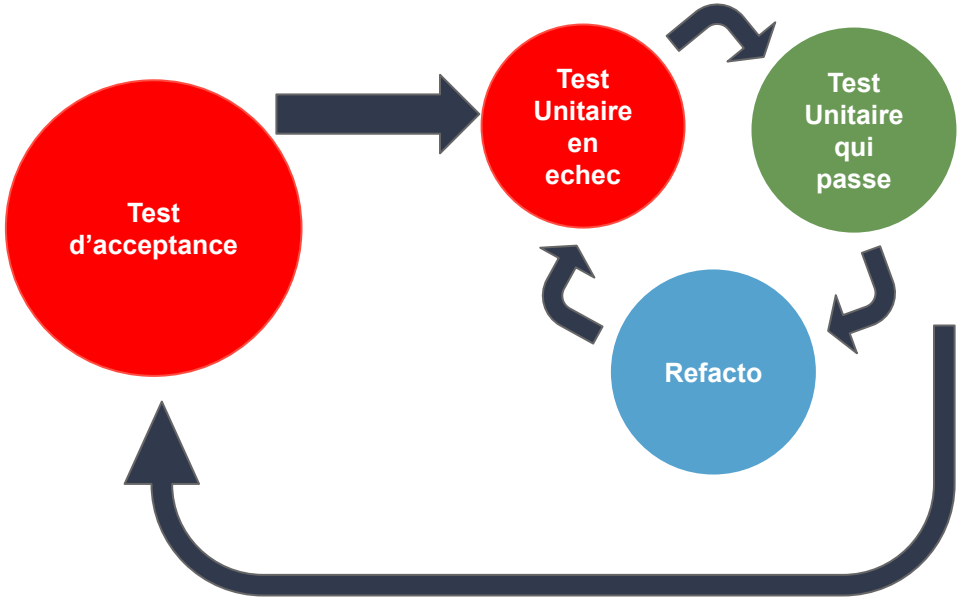
Exercice : "Supertest"

30 minutes

- Reprenez l'exercice Météo avec Swagger et Axios
- Remaniez votre code (Refacto) afin d'exporter l'application Express sans pour autant lancer le serveur en mode écoute (listen)
  - Créer un fichier `app.ts` pour y déplacer l'application Express
  - Conservez dans le fichier `index.ts` le démarrage en écoute sur l'app (listen)

- Créer des tests d'intégrations à l'aide de **supertest** en important l'app express
  - Testez votre appel api

Outside In TDD



L'approche outside in TDD consiste à partir d'un besoin utilisateur, un test d'acceptance, souvent un appel API puis :

- A commencer par écrire un test d'intégration (Supertest) de l'appel API qui ne passera pas car aucune logique n'existe encore
- D'implémenter alors la logique en TDD en commençant par écrire des tests unitaires pour chaque cas identifié
- Un fois que l'on pense que la logique est implémentée, on relance le test d'intégration pour voir si il passe au vert …

Exemple : Pour le jeu du Tic Tac Toe (Morpion) le besoin utilisateur est d'implémenter le endpoint qui permet à l'ordinateur de proposer son mouvement de jeu (marquer une case avec un 2)

- Implémenter un test d'intégration avec un appel API GET /computer-mark en partant du damier [2,0,1,0,1,0,2,0,1]
- Implémenter des règles de décision de choix de positions en TDD (Si un mouvement est possible l'effectuer, Si l'adversaire à un mouvement gagnant le bloquer, ….)
- Continuer la boucle …

Exercice : "Backend Api Tic Tac Toe" en Outside In TDD

1 heure

- Créer trois endpoints
  - POST http://localhost:3000/new-game
  - PUT /mark-player/:cellNumber
  - GET /get-computer-mark
- Il doivent tous renvoyer un json contenant un tableau de 9 position nommé board et pouvant contenir uniquement les valeurs suivantes
  - 0 : case vide
  - 1: case marquée par le joueur (vous)
  - 2: case marquée par l'ordinateur
- Le endpoint get-computer-mark doit être fait en Outside In TDD et il doit permettre à l'ordinateur de jouer la position qu'il considère la plus adaptée pour gagner ou pour se défendre
- Vous pouvez vous inspirer des description de tests suivants :
  - Si un mouvement gagnant est possible, effectuez-le.
  - Si l'adversaire a un mouvement gagnant, bloquez-le.
  - Si le centre est libre, prenez-le.
  - Prenez n'importe quel mouvement disponible.
- Une fois terminé testez le avec **Swagger** puis avec ce client **NextJS** :
  https://ippon-node-course-assistant.fly.dev/exercices/tic-tac-toe
  Dans ce cas vous devez mettre en place cors sur votre serveur express

```
npm install cors
npm install -D @types/cors
import cors from 'cors'
app.use(cors());
```

06 — Bases de données et ORM

Prisma, TypeOrm or Sequelize ?

Dans l’écosystème NodeJS, il existe 3 ORM principaux qui sont Prisma, Sequelize et TypeORM

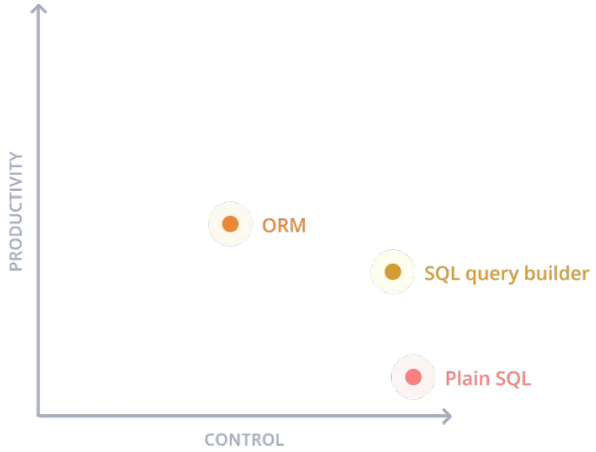
- Prisma connaît un franc succès autour d’une grande communauté pour sa facilité d’utilisation (DX) et son support natif de typescript (Modélisation intuitive, Migrations très simples, Prisma Studio…)

- Sequelize est le plus ancien des trois, il a pu connaître des périodes de transition difficiles entre certaines versions mais c’est apparemment du passé, son support complet de typescript est en "work in progress", mais il reste néanmoins un très bon ORM très populaire

- TypeOrm quand à lui même si il reste un bon ORM, est plus complexe à utiliser, possède une syntaxe bien plus verbeuse (proche de Jpa) et à un très faible support de typescript, néanmoins cela reste un ORM de qualité

Les problèmes avec SQL, les ORMs et autres outils

Les outils de base de données existant actuellement dans l'écosystème Node.js et TypeScript nécessitent un compromis important entre productivité et contrôle.



- Raw queries (Plain SQL)
  - Contrôle total sur les opérations en base de donnée
  - Complexe : RDBMS et NOSQSL ont des opérations différentes
  - Résultats non type safe (Typage manuel → refacto en cas de changement au niveau du schéma)
  - Zéro auto complétion IDE

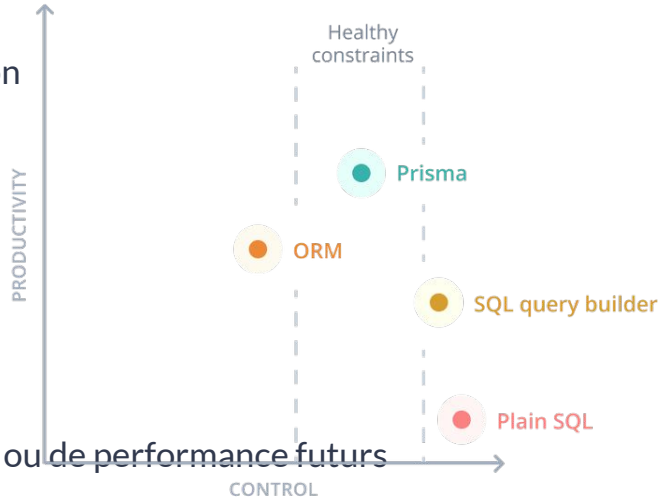
- SqlQueryBuilders (Knex.js)
  - Contrôle élevé, productivité moyenne
  - Ne marche qu'avec les RDBMS
  - Auto completion, aide de l'IDE, plus facile d'écrire des requêtes qu'avec du Plain SQL
  - On continue de réfléchir en mode Table et pas en mode objet

```
knex('users').join('contacts', 'users.id', '=', 'contacts.user_id')  .select('users.id', 'contacts.phone')
```

- ORM
  - Moins de contrôle, plus de productivité
  - Plus proche du modèle mental des développeurs, on pense en terme d'objets et pas de SQL
  - n+1 problem
  - Jointures coûteuses

Let’s go with Prisma !

- Prisma part du constat que les développeurs devraient pouvoir utiliser une abstraction qui prend les bonnes décisions pour eux (en imposant certaines "Healthy Constraints" qui les aident à ne pas faire d’erreurs
  Belle promesse en tout cas 😁 !
- Prisma est le seul ORM entièrement type-safe dans l'écosystème TypeScript.
  - Il permet de produire des requêtes TypeSafe validée à la compilation
  - Il permet de penser en terme d’objets
  - Il permet une excellente auto complétion
  - Queries not classes : Prisma met l’accent sur des requêtes simples et directes et pas sur des classes
  - Schéma Prisma : Référence unique à la fois pour la structure de la base et pour les modèles d’application
  - Healthy Constraints : Prisma incorpore des bonnes pratiques et de design patterns qui protègent contre les problèmes de design ou de performance futurs



Installation et Mise en place

- `npm i -D prisma` → Installation de Prisma en dev dependencies (écrit en typescript donc pas de types)
- `npx prisma init --output=generated/client --datasource-provider mongodb` → Génère un fichie prisma/schema.prisma
- `.env` → Configurez votre fichier .env pour pointer vers une instance **docker (rancher)** ou **https://cloud.mongodb.com/**
- Configurer le schéma **prisma/schema.prisma**
- `npm i @prisma/client` → Installer le client prisma, cela invoque automatiquement `prisma generate` , qui génère un client correspondant au schéma
- `npx prisma db push` → Répercute les modification du schéma en base de donnée et régénère le client

Intégration du client dans le serveur express

Prisma permet la génération de votre client à partir de votre schéma, depuis le serveur express, il est possible d'utiliser directement le client généré et il faut aussi s'assurer de le déconnecter proprement lors de l'arret du serveur (Graceful Shutdown)

```
function shutDown() {
  console.log('Received kill signal, shutting down gracefully')
  server.close(async () => {
    console.log('Closed out remaining connections')
    await prismaClient.$disconnect()
    process.exit(0)
  })
  setTimeout(() => {
   console.error('Could not close connections in time, forcefully shutting down')
    process.exit(1)
  }, 5000)
}

process.on('SIGTERM', shutDown) // kill
process.on('SIGINT', shutDown)  // Ctrl+C in terminal
process.on('SIGUSR2', shutDown) // nodemon restart
```

index.ts

```
import { PrismaClient } from 'prisma/generated/client'

export const prismaClient = new PrismaClient()
```

app.ts

Exercice : “Un petit post facile avec la DB”

30 minutes

- Voici un modèle Prisma pour une base de donnée Mongo
  - Vous pouvez créer une base mongo gratuite ici : https://cloud.mongodb.com/
  - doc prisma schema models : https://www.prisma.io/docs/concepts/components/prisma-schema/data-model
- Récupérez les éléments du schéma ci-dessous et intégrez les dans votre schéma Prisma Mongo et régénérez le client, le tout à l’aide de la commande suivante : `npx prisma db push`

```
model Post {
  id String @id @default(auto()) @map("_id") @db.ObjectId
  title String
  body String
  author User @relation(fields: [authorId], references: [id])
  authorId String @db.ObjectId    }

model User {
  id String @id @default(auto()) @map("_id") @db.ObjectId
  email String @unique
  name String?
  posts Post[]    }
```

- Créér une route /new-user avec un verbe Post et créér l’utilisateur et son post tel que ci-dessous :

```
prismaClient.user.create({
  data: {
          name: 'Rich', email: 'hello@prisma.com',
          posts: {
                    create: { title: 'My first post', body: 'Lots of really interesting stuff'}
                  }
      }
})
```

Mocker ses dépendances dans les tests d'intégrations

- Si vous avez souhaité mettre en place un test d'intégration avec l'appel Api POST de l'exercice précédent, vous vous êtes probablement rendu compte qu'il vous fallait une base de donnée pour que votre test marche
- Par ailleurs, si vous appelez deux fois votre Api, le second appel échouera car la propriété User.email du schéma (et donc de la colonne correspondante dans base de donnée) à une contrainte unique
- Prisma offre une possibilité de mocker le framework lui même mais pour ma part je recommande l'approche d'inversion des dépendances et l'ajout d'une abstraction de type repository par exemple

```
beforeAll(async () => {
      repositoriesMock = {
            mongoRepository: {
                  createUser: vi.fn()
            },
            postgresRepository: {}
      }
      app = appProvider(repositoriesMock)
})
```

Typologies de Test : Exemple d'un test d'integration avec l'api

- Lorsque l'on tests des APIs, on ne veut pas de coupler à la base de donnée
- Pourtant les APIs font souvent appel aux dépendances de type DB

- L'utilisation de mocks et du typage typescript vous permet de vérifier la **collaboration** entre votre repository et votre api
- la méthode **"createProject"** reçoit **"name"** en paramètre et le dernier **expect** vérifie que le paramètre **"name"** envoyé à la route **POST** est bien passé à la méthode **"createProject"** du repository.

```
import { expect, describe, test, vi, beforeAll } from 'vitest'
import request from 'supertest';
import appProvider from '../app'
import { repositories } from '../repositories';
import { Express } from 'express'

let repositoriesMock: repositories, app: Express

Run | Debug
describe(`Tests d'API`, () => {
    beforeAll(async () => {
        repositoriesMock = {
            mongoRepository: {
                createProject: vi.fn((name:string) => Promise.resolve(
                    { id: "65451c26ac6932248cad1543", name })),
                createTicket: vi.fn()
            },
            postgresRepository: {}
        }
        app = appProvider(repositoriesMock)
    })
    Run | Debug
    test('should return 201 OK for /kanbanProject', async () => {
        // When
        const response = await request(app)
            .post('/kanbanProject')
            .send({ name: 'Antares VI' })

        // Then
        expect(response.status).toBe(201);
        expect(response.type).toBe('application/json');
        expect(response.body).toHaveProperty('id', "65451c26ac6932248cad1543");
        expect(response.body).toHaveProperty('name', "Antares VI");
    });
});
```

Typologies de Test : Exemple d'un test d'intégration avec la DB

- Les méthodes du repository qui effectuent des actions dans la base de données ont néanmoins besoin d'être testées
- Ces tests garantissent que le schéma est à tout moment compatible avec les interactions de l'entièreté votre application avec la base de donnée
- Les **repositories** constituent des abstractions qui encapsulent les interactions avec la DB
- Ces types de tests sont plus coûteux, plus long à réaliser et nécessitent une ou plusieurs DB
- On utilise généralement des images docker que l'on instancie généralement dans le script "test:database"
- On nettoie la base à chaque teardown et on ne parallélise **jamais** ces tests

```
import { expect, describe, test, afterAll } from 'vitest'
import appProvider from '../app'
import { databaseRepositories, mongoPrismaClient } from '../repositories';
import { beforeEach } from 'node:test';
import { Status } from '../../database-mongo/prisma/generated/client-mongo';

const app = appProvider(databaseRepositories)
Run | Debug
describe(`Tests de Repositories : mongoRepository`, () => {
    beforeEach(async () => {
        await mongoPrismaClient.project.deleteMany()
        await mongoPrismaClient.ticket.deleteMany()
    })
    afterAll(async () => {
        await mongoPrismaClient.project.deleteMany()
        await mongoPrismaClient.ticket.deleteMany()
    })
    Run | Debug
    test('should create a project and tickets related to it in the database', async () => {
        // When
        const { mongoRepository } = databaseRepositories
        const project = await mongoRepository.createProject("Antares XVI")
        const ticket_1 = await mongoRepository.createTicket(
            { name: 'Plongée pour inventaire de coraux ', status: Status.TODO, projectId: project.id })
        const ticket_2 = await mongoRepository.createTicket(
            { name: 'Étude acoustique de chants de cétacés ', status: Status.DONE, projectId: project.id })

        const projectFromDb = await mongoPrismaClient.project.findUnique({
            where: { id: project.id },
            include: { tickets: true }
        })

        // Then
        expect(projectFromDb).toStrictEqual({ ...project, tickets: [ticket_1, ticket_2] });
    });
});
```

Exercice : "Backend Kanban"

1 heure

- Créer trois endpoints qui permettent de gérer des projets Kanban dans une base de donnée
  - POST http://localhost:3000/kanbanProject (on passe un paramètre name:string dans le body)
  - GET http://localhost:3000/kanbanProject (on récupère un kanban et ses tickets)
  - POST http://localhost:3000/ticket (on passe les paramètres name:string et status:enum dans le body)
  - DELETE http://localhost:3000/ticket/{id}

- Le projet Kanban doit contenir les concepts suivants
  - id : alphanumérique auto généré unique et obligatoire
  - name: alphanumérique unique et obligatoire
  - tickets: un ensemble de tickets relatifs à ce projet

- Le Ticket doit contenir les concepts suivants
  - id : alphanumérique auto généré unique et obligatoire
  - name: alphanumérique unique et obligatoire
  - status : état du ticket dont les valeurs peuvent prendre (TODO, DOING, DONE)

- Il faut tester (TDD) !

- Une fois terminé testez le avec **Swagger** puis avec ce client **NextJS** :
  https://ippon-node-course-assistant.fly.dev/exercices/kanban
  Dans ce cas vous devez mettre en place cors sur votre serveur express

```
npm install cors
npm install -D @types/cors
import cors from 'cors'
app.use(cors());
```